

clauses like FROM, WHERE, GROUP BY, ORDER BY, LIMIT, INTO, etc.

Let's see some examples of time series specific queries:

```
SELECT min(value),max(value),mean(value) FROM temperature
  WHERE city='pune'and time > now()-6h GROUP BY time(10m)
SELECT min(value),max(value), mean(value) FROM temperature
  WHERE time > now()-6h GROUP BY city,time(10m) fill(none)
SELECT min(value),max(value), mean(value) FROM temperature
  WHERE time > now()-6h GROUP BY time(10m) fill 0
SELECT *FROM temperature WHERE time>now()-15h ORDER BY time DESC
```

In the first query, data is queried for the past six hours with a filter of Pune as the city and grouping the aggregates every 10 minutes. In the second query, instead of filtering by city we are grouping by city, and skipping the results with empty values. In the third query, empty values are replaced by a default value like zero. The GROUP BY time clause requires the WHERE time clause, and it's obvious that the GROUP BY interval exceeding the time slot in the WHERE clause is not meaningful. The WHERE time clause can be based on absolute timestamps also. The last query provides results in the descending order of timestamps, which fall over the last 15 hours.

Transforming data into other series

One can store the results of a query into other measurements or series. For example:

```
SELECT mean(value) INTO avg_temperature FROM temperature
  WHERE time > now() - 6h
SELECT mean(value) INTO city_wise_temperature FROM
  temperature GROUP BY city
```

Downsampling of data

Since data volume is high in IoT and DevOps scenarios, it's not feasible to retain all the data for a long time. So one can periodically take aggregations, anomalies from original series, store in long term series or other forms, and flush the original series. InfluxDb comes with downsampling of high precision data using the following two features.

 **Note:** Here, precision refers to data volume and time frequency, and not the accuracy of floating values.

Retention policies: Each database comes with retention policies, which decide the persistence of data in measurements stored under it. By default, 'autogen' is the retention policy with infinite duration, i.e., data is never flushed out from autogen measurements. We can create custom retention policies for the desired duration, and select one of the policies as default per database.

Here is the syntax for creating a retention policy:

```
CREATE RETENTION POLICY <rp-name> ON <db-name> DURATION
<duration>REPLICATION<n>[SHARD DURATION<duration>][DEFAULT]
```

Please refer to Storage Engine and Glossary of Terms sections of the documentation for more details on storage engine, shard groups, shard duration, etc.

Let's create a few retention policies for the durations of one year, three days, two hours and call them long-term, mid-term and short-term policies.

```
CREATE RETENTION POLICY rp_lterm ON cityweather DURATION 365d
  REPLICATION 1
```

```
CREATE RETENTION POLICY rp_sterm ON cityweather DURATION 2h
  REPLICATION 1
```

```
CREATE RETENTION POLICY rp_mterm ON cityweather DURATION 72h
  REPLICATION 1 SHARD DURATION 3h
```

Typically, unqualified measurements are considered under a default retention policy. While writing data or querying, measurements can be qualified with non-default policies such as `<policy-name>.<measurement>` -- for example, `rp_lterm.temperature`, `rp_sterm.humidity`, `autogen.pressure`, etc.

Transformed data can be stored in other databases also, using fully qualified names in the form of `<db-name>.<rp-name>.<measurement>` -- for example, `weatherdb.autogen.aggr_temperature`, `weatherdb.aggr_temperature`, etc.

To list out all policies or drop a particular policy, you can use the following syntax:

```
SHOW RETENTION POLICIES on <db-name>
DROP RETENTION POLICY <rp-name> on <db-name>
```

Continuous queries: Before flushing the data, we need to take out aggregations and data beyond thresholds, and store the resultant points in measurements under a long-term retention policy or autogen. For this, one can schedule a query for periodical execution. This is possible through continuous queries. Let's consider a few examples:

```
CREATE CONTINUOUS QUERY "cq_temp" ON "cityweather"
BEGIN
  SELECT MEAN("value"),MIN("value"),MAX("value") INTO
  autogen.aggr_temperature FROM rp_short_term.temperature
  WHERE city='pune' GROUP BY time(1m)
END
CREATE CONTINUOUS QUERY "cq_humidity" ON "cityweather"
BEGIN
  SELECT * INTO rp_long_term.beyond_humidity FROM rp_
  short_term.humidity WHERE time > now()-1h humidity > 90
END
```